
FAQ: Latency

Version 3.0 January 9, 2024

List of Questions

Q1	How can a Transaction Fabrix (TF) have a lower latency than a ‘flat’ Clos Network?	3
Q2	How can this be if there are say 19 hops from one side of the row to the other?	3
Q3	Who cares about latency that much?	3
Q4	Why is Latency the Most Important problem to solve?	4
Q5	Why not deploy ECN?	5
Q6	What kind of Latency goals do you imagine, for example, with 5G?	5
Q7	Why are the cable latencies so much lower for the TF than for a conventional Clos?	5
Q8	How does the Transaction Fabrix compare to the latency of a Clos network?	5
Q9	Can’t we do this trick in Conventional Datacenters?	6
Q10	But can’t switches also do cut-through?	6
Q11	How do you measure switch latency?	6
Q12	Why are Kleinberg/Huang Links relevant?	6
Q13	How fast can a cell do forwarding of packets (with our without Kleinberg/Huang Links)? .	6
Q14	How do you compare to FreeFlow?	6
Q15	What Else is there to consider about Latency?	7
Q16	What about FreeFlow’s claims to low latency using Zero copy?	7
Q17	How does the Transaction Fabrix manage its communications plane?	7
Q18	What’s the difference between conventional orchestrators and the TF orchestrator?	7
Q19	How do you calculate the forwarding delay for a minimum packet in the SmartNIC?	8
Q20	How can software routing times be so fast when hashing takes hundreds of instructions? . .	8
Q21	What are the benefits of direct connections?	8
Q22	Can you use Supermicro’s Big Twin 2U2N config?	8
Q23	What other configurations are possible?	9
Q24	Can you use the Facebook OCP 2U3N configuration?	9
Q25	Can you use the LinkedIn Open19 1U4N configuration?	9
Q26	What would look like with regular 1U 1N Pizza boxes?	9
Q27	Why can’t you just use faster links, e.g. 40 or 100Gbit ports to unclog the pipeline?	9
Q28	What about 100 and 200 Gbit interfaces that are coming this year?	9
Q29	What do Kleinberg links look like with 1U servers (1U1N) ?	10
Q30	So your minimum cable hopcount is 6 or 7?	10
Q31	So then your average hopcount is 6 or 7?	10
Q32	Why do you refer to the Clos Network as a ‘choke point’?	10
Q33	Why not use flat addressing like a Clos network?	10
Q34	What’s wrong with that assumption of random placement?	10
Q35	What is ‘Tiled Tree Paxos’?	11
Q36	On Paxos ... why are 9 nodes an important cluster size?	11
Q37	Don’t you need multiple endpoints with separate addresses to manage things like failover? .	11
Q38	How many hops are in a large datacenter?	11
Q39	What if some functions cannot talk directly with 1-hop (logically adjacent) relationships? .	11
Q40	What about scalability?	12
Q41	How does Transaction Fabrix compare with switch latency?	13
Q42	What about layer 1 or physical layer switches?	14
Q43	What are the advantages do you get from the TF besides eliminating the TOR switch latency?	14
Q44	But isn’t forwarding in software always going to be slower than hardware?	14
Q45	What about Cut-Through Switching? Aren’t switches better at this than a typical Smart-NIC?	15

Q46	But what about long packets, and the bit transfer time on the wire?	15
Q47	What if ‘something unexpected’ in provisioning functions on the fabric require multiple hops?	15
Q48	What size packets are needed for transactions?	16
Q49	What is Tail Latency?	16
Q50	Why are you so obsessed with microseconds (as opposed to nanoseconds or milliseconds?)	16
Q51	How do you calculate the wire propagation delay exactly?	16
Q52	Doesn’t the low valency of a cell limit the reachability of cells compared to a high radix switch?	16
Q53	How long does it take to send a packet?	17
Q54	Doesn’t your Liveness (ENTL) packets interfere with normal packet transmission?	18
Q55	If temporal intimacy requires a small amount of information, why use a whole packet?	18
Q56	What are Huang Links?	18
Q57	What is the use case for these Kleinberg/Huang Links?	19
Q58	How can you forward a packet through a cell faster than a switch can?	20
Q59	What about PingMesh?	20
Q60	What about TIMELY	21
Q61	How do you compare with the Google View (Killer Microseconds)	21
Q62	How do you compare with the EPFL View (Killer Microseconds)	22
Q63	How do you compare to Virtual Synchrony?	22
Q64	How do you compare to other Low Latency approaches in the literature?	22

The Questions

[Q1] How can a Transaction Fabrix (TF) have a lower latency than a ‘flat’ Clos Network?

[A1] The communications structure for microservices, or deep learning is *not* flat, it’s a *graph*. Imposing a ‘flat’ Any-to-Any (A2A) structure *for functional packets* is no longer relevant when microservice communications are dominated by *functional* East-West traffic, and not North-South *transit* traffic.

[A1] Because even if computational entities are *randomly* placed, we have *no more* hops than a Clos Network. When they are not randomly placed, we have a *single* hop between each computational entity.

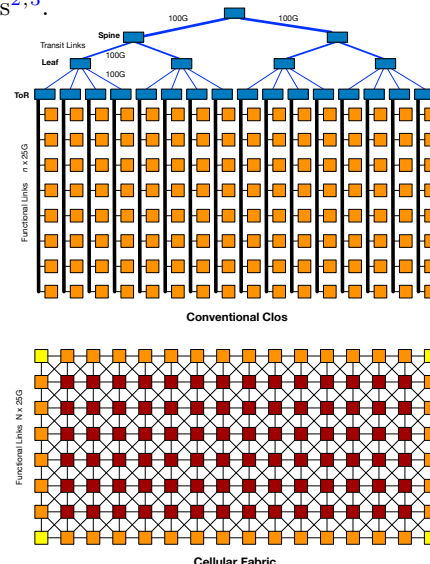
[A1] Even with present Clos networks, locality is significant: most locality happens ‘by accident’. I.e., Datacenter provisioners already allocate subsystems that need to coordinate in the same rack. Measurements show that 75% of traffic for cloud infrastructures are within the same rack. Trying to preserve the principle of a ‘flat’ network is an illusion that network engineers want. This is *not* what developers want or need. Developers just want their packets to their destination(s) as fast as possible, in-order.

[A1] With our tree addressing we can migrate with less management overhead (see FAQ-Addressing).

[A1] The table on the next page shows an 800+ cells (20 rack row of 42 cells each). We calculate the cable latencies plus hop (switching) latencies, assuming that transit time through a cell is $100ns$. Example: a (first-bit)¹ Transit path through the fabric: 9 cells (the maximum diameter for this TF + 10 cables = $900ns$ for switching + $22ns$ cables = $922ns$ → Less than $1\mu s$ for > 800 cells^{2,3}.

Conventional (Clos) Network Example. The table (next page) shows an analysis for an 800 server (40×20) configuration (ToR’s take up 2-4 spaces in the rack) and 3-tiers of switches (ToR, Leaf, Spline). Links in blue between the switches are wasted – they carry only *transit* packets which consume link [Bandwidth](#) with redundant traffic. Clos is also limited in its path redundancy (ECMP tries to balance the few paths available).

Transaction Fabrix Example. The table also shows an 800+ cells (42×20) configuration. Because the Transaction Fabrix uses tree provisioning, and active migration of computational entities, the red links carry mostly functional packets, and are most often 1 hop away. When using Kleinberg links, even with random placement the maximum hopcount is $\text{Log}(N)$, and the average is $\text{LogLog}(N)$.



[Q2] How can this be if there are say 19 hops from one side of the row to the other?

[A2] In this analysis we account for Kleinberg⁴ effects by taking $\text{LogLog}(N)$ as average hopcount and rounding up the number of cells serving as switches, and $\text{Log}(N)$ for the maximum hopcount⁵.

[Q3] Who cares about latency that much?

[A3] Anyone who cares about performance. e.g. applications such as OLTP are notoriously latency sensitive.

[A3] Anyone whose software interacts with human beings. User requests that must complete in 1 sec might invoke 100 usercalls; leaving only 10ms for each including compute time.

[A3] For Google: [predictable latency is everything](#) (implies ‘not low’ – most likely because they can’t achieve that in their network). Transaction Fabrix can achieve predictable *and* theoretically lowest possible latency.

¹Full packets also require transmission time on the wire, and varies with cut-through or store and forward (analyzed later).

²We give switches the benefit of the doubt, by assuming a $50ns$ latency through a hardware switch and $100ns$ latency through a Cell (in firmware in the NIC). Ultimately, there is no reason that the cell latency could not be as good as a switch.

³c.f. In [Google’s current infrastructure](#), best real-world latencies are $10\mu s$, i.e. $10 \times$ larger.

⁴References - Kleinberg Link paper that proves $\text{LogLog}(N)$ Average and $\text{Log}(N)$ Max.

⁵We could round down in this analysis, but don’t need to be that aggressive to prove our point.

Cellular Fabric Latency

Servers Per rack n	Racks per Row m	Total Servers $n*m$	Func. Links <i>See ref</i> $*$	Minimum Hops (ns)	Average Hops LogLogN (length)	Maximum Hops LogN (length)	Minimum Hops (Transit)	Average Hops (Transit) LogLogN	Maximum Hops (Transit) LogN	Minimum Latency Switches+ Cables	Average Latency Switches+ Cables	Maximum Latency Switches+ Cables
			Length(m):	All Func.	ns	ns	All Func.	(round up)	(round up)	ns	ns	ns
	20		Latency(s):	2.2	2.2	2.2	100	100	100			
2	20	40	96	2.2	2.4	5.3	1	7	5	124.8	705.3	511.7
4	20	80	250	2.2	2.7	6.3	1	2	6	124.8	205.9	613.9
6	20	120	404	2.2	2.8	6.9	1	2	6	124.8	206.1	615.2
8	20	160	558	2.2	2.9	7.3	1	2	7	124.8	206.3	716.1
10	20	200	712	2.2	2.9	7.6	1	2	7	124.8	206.5	716.8
12	20	240	866	2.2	3.0	7.9	1	2	7	124.8	206.6	717.4
14	20	280	1020	2.2	3.0	8.1	1	3	8	124.8	306.7	817.9
16	20	320	1174	2.2	3.1	8.3	1	3	8	124.8	306.7	818.3
18	20	360	1328	2.2	3.1	8.5	1	3	8	124.8	306.8	818.7
20	20	400	1482	2.2	3.1	8.6	1	3	8	124.8	306.8	819.0
22	20	440	1636	2.2	3.1	8.8	1	3	8	124.8	306.9	819.3
24	20	480	1790	2.2	3.2	8.9	1	3	8	124.8	306.9	819.6
26	20	520	1944	2.2	3.2	9.0	1	3	9	124.8	307.0	919.8
28	20	560	2098	2.2	3.2	9.1	1	3	9	124.8	307.0	920.1
30	20	600	2252	2.2	3.2	9.2	1	3	9	124.8	307.1	920.3
32	20	640	2406	2.2	3.2	9.3	1	3	9	124.8	307.1	920.5
34	20	680	2560	2.2	3.2	9.4	1	3	9	124.8	307.1	920.7
36	20	720	2714	2.2	3.2	9.5	1	3	9	124.8	307.1	920.9
38	20	760	2868	2.2	3.3	9.6	1	3	9	124.8	307.2	921.1
40	20	800	3022	2.2	3.3	9.6	1	3	9	124.8	307.2	921.2
42	20	840	3176	2.2	3.3	9.7	1	3	9	124.8	307.2	921.4
				Min	Ave	Max	Min	Ave	Max	Min	Ave	Max
				Hops (cables) (ns)			Hops (switches)			Total Latency (ns)		

[A3] The key is to not focus on the minimum latency, we have that nailed anyway by eliminating switches from the path from one server to another. The issue for distributed systems trying to communicate in today's datacenters is the tail latency, which in general is unbounded (which is what people confuse with partitioning in the form of the CAP Theorem).

[Q4] Why is Latency the Most Important problem to solve?

[A4] Because “bloated buffers lead to network-crippling latency spikes” [bufferbloat.net] “This is a classic ‘black swan’ situation in Nassim Taleb’s sense; in today’s Internet-dependent economy there is a potential for nearly incalculable havoc in the worst case, but we don’t even know in principle how to estimate the overall risk. Bufferbloat mitigation might keep us out of some very serious trouble, and is worth pursuing on those grounds alone.”

[A4] “TCP is an end to end servo system, if you don’t respond in a timely manner, you defeat congestion avoidance mechanisms”. [Jim Gettys, Google Talk]

[A4] The problem is that ‘retry oriented’ protocols will create cascade failures that show up as congestion, dropped packets, and tail latency. Daedalus comes at this from a very different perspective: first, we don’t drop packets (deliberately). Second, for certain important packets (e.g. transactions), we unsend (pull the packet back before sending more packets). This form of logical reversibility puts the problem of buffering and bufferbloat back in the originating application, providing a natural form of flow control that eliminates the problem.

[Q5] Why not deploy ECN?

[A5] **Explicit Congestion Notification (ECN)** allows end-to-end notification of network congestion without dropping packets. Conventionally, TCP/IP networks signal congestion by dropping packets. When ECN is successfully negotiated, an ECN-aware router may set a mark in the IP header instead of dropping a packet in order to signal impending congestion. The receiver of the packet echoes the congestion indication to the sender, which reduces its transmission rate as if it detected a dropped packet.

[A5] The problem is that ECN is optional, and it must be supported by the underlying network infrastructure. The Transaction Fabrix protocols can tunnel conventional TCP (and UDP) packets, and we will block the sender from overloading the network. We can also signal

[A5] Solving this problem (i.e. simultaneously lowering latency, increasing resilience and simplifying) requires a change in perspective. We are going from a forward in time “bang bang bang” perspective on networking, to a logically reversible form of communication which naturally subsumes flow control, and provides, for the first time, the ability to perform ‘exactly once’ semantics between two or more endpoints.

[Q6] What kind of Latency goals do you imagine, for example, with 5G?

[A6] 5G currently assumes 10ms is an adequate goal. This is uninteresting. Edge applications can exploit human sensitivity at 1ms latency for AR/VR. We suspect this is because of the architectural/technology box they are thinking in. 3.3 microseconds is 1 Kilometer at the speed of light. Granted, this is in free space. But even in fiber (which is slower than copper) its still 2/3 of that. Microseconds should be the 5G asymptotic goal at the edge. At least for minimum latency. Long tail latency truncated at 1ms will then become feasible (when we think outside this box).

[Q7] Why are the cable latencies so much lower for the TF than for a conventional Clos?

[A7] The first hop is nothing but a one cable delay; there are no cell (or switch) forwarding delays for 1 hop.

[A7] In general, Transaction Fabrix will have far shorter cables. They connect to their neighbors a tenth to a half a meter away (in the same rack), instead of some switch box half a row (10s of meters) away.

[A7] Transaction Fabrix replaces store and forward in conventional datacenters with genuine cut-through routing.

[Q8] How does the Transaction Fabrix compare to the latency of a Clos network?

[A8] All links in a conventional Clos are dominated by the bit transmission times T_{bt} , and thus most often revert to store and forward (S&F) routing.

[A8] Whatever way you look at it: with switches, the *minimum* number of hops between two servers cannot be less than two cable hops and one switch hop. In addition, switches have their own inherent latency (S_L)⁶. So the total server to server latency L_{ss} would be:

$$L_{ss} = \sum_{1..Nhops} H_l \cdot c_{med} + \sum_{1..N-1} S_L + \sum_{1..N-1} T_{bt} + \sum_{1..N-1} T_{sf} \quad (1)$$

Where H_l = length of each hop in meters, c_{med} = speed of light in the medium in m/s (fiber for Conventional Data Centers (CDC’s) and copper for Transaction Fabrix, where cables are much shorter). T_{bt} is the bit transmission time, and T_{sf} is the penalty paid having to store and forward. With a Transaction Fabrix, the situation is very different, the minimum (and most likely) number of hops between two servers (cells) is *one* cable hop and *zero* cell hops.

In the Transaction Fabrix, a single logical hop is just a single cable (~ 2 ns), whereas a single logical hop in a Clos, has to go through the Top of Rack switch and two cables – incurring two cable delays, a switch

⁶The best 100 Gb/sec InfiniBand switches (including Intel’s Omni-Path as well as Mellanox Technologies Switch-IB) have around 200 nanoseconds of port-to-port latency, and the average 100 Gb/sec Ethernet switch is in the range of 300-400 nanoseconds. The Gen-Z consortium claims that Gen-Z switches could be on the order of 3X to 13X faster.

delay, and two sets of bit transmission times T_{bt} . Because, in general, even cut-through capable TOR's have to revert to store and forward in the mixed environments of typical datacenters.

[Q9] Can't we do this trick in Conventional Datacenters? So that microservices that need to talk to each other are provisioned in the same rack under the same TOR Switch?

[A9] Not without a great deal of manual effort and operational complexity. The key impediment is that in Any-to-Any (A2A) addressing (with Ethernet or IP) addresses have to change when entities migrate. In our Neighbor to Neighbor Addressing (N2N) Fabric, entities are free to migrate anywhere on the tree⁷, and the routes for traffic intended for them automatically follows them without any need to change anything.

[Q10] But can't switches also do cut-through?

[A10] Switch latency is the amount of time a packet spends traversing the switch. While simple in the abstract, in practice actual latencies through a switch can vary enormously – by potentially by orders-of-magnitude⁸.

[Q11] How do you measure switch latency?

[A11] Switch latency is measured from port-to-port on an Ethernet switch. It may be reported in a variety of ways, depending on the switching paradigm that the switch employs: Cut-through or Store-and-Forward.

Switch latency is the amount of time that an Ethernet packet spends traversing an Ethernet switch. While simple in concept, in practice there are a number of factors that can cause the actual latency that a packet experiences to vary by a large amount, potentially by orders-of-magnitude. Switch latency is measured from port-to-port on an Ethernet switch. It may be reported in a variety of ways, depending on the switching paradigm that the switch employs: Cut-through or Store-and-Forward.

[Q12] Why are Kleinberg/Huang Links relevant?

[A12] Kleinberg links are random length cables which enable 6 degrees of separation. Huang links are direct connects between the in and out ports of a single cell – which enables virtual Kleinberg links on the Transaction Fabric. Huang links are pre-programmed by the infrastructure owner to provide direct (secure) cut-through paths, primarily for the purposes of confinement (security), but which have the additional benefit of much lower latency, and no congestion on the output links (i.e. solves the incast problem, at least on links).

[Q13] How fast can a cell do forwarding of packets (with or without Kleinberg/Huang Links)?

[A13] There are several kinds of forwarding possible (depending on the implementation):

- Physical Huang Links (e.g. through jumpers or software controlled relays) [~ 1 ns added to the path]
- Bit cut through in the NIC hardware (logic forwards each bit on an input wire to a preselected output wire) [~ 5 ns]. This is not currently a feature of most NIC's however, it is an easy feature to add.
- Forwarding through the Cell Agent. (see calculation below)
- Hashed store & forward through the forwarding engine [200 instructions, $\sim 1\mu s + \text{Buffering Time}$ ⁹]
- Indexed store & forwarding through forwarding engine [10 instructions, $\sim 500ns + \text{Buffering Time}$]
- Header cut through through the forwarding engine [10 instructions, $\sim 500ns$]

[Q14] How do you compare to FreeFlow?

⁷Or anywhere period, if the cell is quiesced.

⁸See the whitepapers by [Plexxi](#) and [Cavium](#)

⁹We use memory buffers as communications channels for processes running on the same machine. While doing this in the SmartNIC is the highest performance, even in the device driver we still don't need to traverse the protection zones or invoke application cores when this cell is not the target.

[A14] [FreeFlow](#) and the Transaction Fabrix (TF) are complementary. FreeFlow reduces latency *inside* computers. The TF reduces latency (and improves reliability) *between* computers, the two have additive benefits.

[A14] We also use [DPDK](#), and derivative technologies such as [ixy](#), to improve the temporal intimacy between applications and the Fabrix.

[Q15] What Else is there to consider about Latency?

[A15] See: [Latency](#) and www.latency.org

[Q16] What about FreeFlow's claims to low latency using Zero copy?

[A16] This is not new. For several years Linux has had APIs through which application code can send and receive packets without ever paying the burden of copying between kernel and application space, and also eliminates a lot of kernel calls due to a shared ring buffer mechanism. It turns out that this isn't really zero copy - deep, deep, deep inside Linux there is a buffer copy (which means that normal Linux packet I/O requires 'two' copies and the ring buffer stuff eliminates one copy).

[Q17] How does the Transaction Fabrix manage its communications plane?

[A17] The Daedaelus LOV orchestrator hides how the messaging is being done (in a similar way to [FreeFlow](#), except that we don't need a central orchestrator). A VM knows which containers are running in it and can short circuit the communications path. Our cell agent knows which VMs are running and can do the same thing.

[A17] Our LOV-orchestrator manages 'stacked trees'. Cell Agents are given membership of an underlying TRAPH from which they may operate. A distinguished cell is delegated ownership from the tree below, and is protected from failure by its consensus tile (up to 3-5-7 or 9 of its surrounding cells). Cell Agents may build stacked trees (subset trees or graph covers), to recursively manage the containers or VM's on their cells. Stacking trees is the process of building relationships between a cell and its children within a single TRAPH. Tree-building is followed by 'rendezvous' with trees after-the-fact. That's controlled by the TRAPH orchestrator. The orchestrator use the GVM, set up some properties and the processes (i.e. creating, or identifying VMs or containers) would use those properties in a predicate to 'join' a tree.

[Q18] What's the difference between conventional orchestrators and the TF orchestrator?

[A18] All current orchestrators use a per service central manager. The LOV Orchestrator (LOV-O) uses purely local information¹⁰. For example, in a sharded key-value store running the LOV-O, should node's shard become too large, it can locally decide to extend the tree and give sub-shards to its children. Should a child find it is holding less than some threshold, it can pass its sub-shard to its parent and drop out of the tree.

Several years ago my friend John Romkey did a project with Van Jacobson to figure out how small they could prune TCP - they got it down to something on the order of 300 machine instructions but only if nothing was out of tune in the packet flow. That ideal packet flow does not exist in real life - or did not. One of the benefits I see of the Daedaelus model (at the mesh level) is that it can deliver an ideal packet flow - which means that a TCP stack, if it knew that the destination was on the same DAE fabric could engage in "fast TCP", i.e. a TCP path in which all of the error checking and handling is not done because it is not necessary.

Which raises a question in my mind - can the Daedaelus fabric eliminate the need for TCP to engage in slow start behavior (or slow-start recovery from congestion detection)? If TCP slow start can be obviated, i.e. if "fast TCP" could start by sending as fast as it can into the open receive window, or even if slow start could be changed to not-quite-as-slow-start, then prodigious overall application latency improvements could occur for those applications (particularly web based ones) that open a lot of short-lived TCP connections.

¹⁰We define *local* information as within the self (current) cell, or information 1 hop away (on a neighbor cell) that is synchronized (using the ENTL/ENTT protocols) and the MetaData Tensor (MDT) with the self-cell

[Q19] How do you calculate the forwarding delay for a minimum packet in the SmartNIC? Consider:

- Transmission time - time from the first bit until the last bit of a message has left the transmitting node
- Propagation delay - time taken by a bit to reach from sender(A) to receiver(B)
- Packet delivery time = transmission time + propagation delay

[A19] Assume: B Gb/s Bandwidth; C CPU clock speed (GHz); M memory access time (in ns); P bytes in packet; H header (overhead) bytes. Forwarding of packet requires K instructions, 1 memory access, 1 packet copy at rate B. If the NIC does header cut through (header delivered before rest of packet arrives), the forwarding time will be: $8H/B + K/C + M + 8P/B$ (wait + processing + memory + copy). B = 25 Gb/s; C = 2 GHz = 0.5 ns/cycle; K = 20 instructions; M = 50 ns (cache hit) (assumes L1 hit); P = 64 B; H = 48 B. Packet forwarding time is 150 ns (300 clocks). Add wire transmission time to the next NIC.

If the NIC does not do cut through, the time is: $8P/B + K/C + M + 8P/B$ or 160 ns for the above figures. Longer memory access times may dominate. For example, M on the Netronome is 300 ns, so add 250 ns to the above numbers. Store and forward delay (non-blocking¹¹):

Total = Receive_Delay + LookUpTime + Send_Delay. Assuming 25GE ports, Netronome chip running 1GHz internal clock. Lookup with Internal SRAM access is around 300 clocks. MTU 1500 Byte Packet receive/forward delay = $(12000 \text{ bit} / 10 \times 10^9) = 1200 \times 10^{-9} = 1200 \text{ ns}$. Lookup with Internal SRAM = $(300 \text{ cycle}) / 1 \times 10^9 = 300 \text{ ns}$. Store-Lookup-Forward Delay = $1200 + 300 + 1200 \text{ ns} = 2700 \text{ ns}$.

- Min size 64Byte packet receive/forward delay = $512 \text{ bit} / 10 \times 10^9 = 51.2 \times 10^{-9} = 51.2 \text{ ns}$.
- Lookup with Internal SRAM = $(300 \text{ cycle}) / 1 \times 10^9 = 300 \text{ ns}$
- Store-Lookup-Forward Delay = $51.2 + 300 + 51.2 \text{ ns} = 402.4 \text{ ns}$

[Q20] How can software routing times be so fast when hashing takes hundreds of instructions?

[A20] Our forwarding scheme uses indexed routing—obviating the need for a hash lookup.

[A20] Forwarding is also fast because it is possible to achieve (a) cut-through routing (don't need to process header) (b), output ports are always available. (c) close to zero transit time through the link (with modified hardware), (d) enhanced security (Huang links are configured by the separation kernel, and thus cannot be reconfigured at run-time to enable snooping on the packets as they go through).

[Q21] What are the benefits of direct connections?

[A21] Our application is highly secure and needs the enhanced privacy of direct connections. We also benefit from lower latency and higher bandwidth by not having a TOR in the paths between them.

[A21] We have an application for some of our customers that require the security of direct (private) connections between servers. Here's an example of 4×4 Half-Bricks within a single Open19 rack. We would like to get as many servers within a single rack as possible (e.g. 96+), hence our interest in half-bricks.

[Q22] Can you use Supermicro's Big Twin 2U2N config?

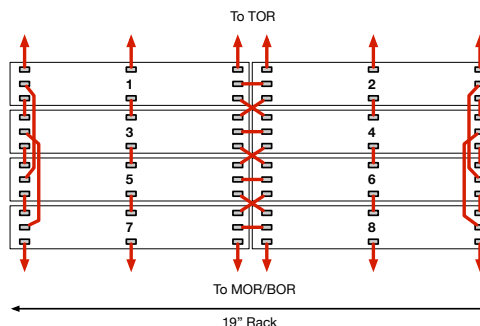
[A22] Yes. Here's the rear panel of this [Big Twin 2U 4-Node](#) unit. Also, Supermicro's new network card interface is ideally suited for our purposes.



¹¹For pure (Huang) forwarding there is no blocking: one input going to one output.

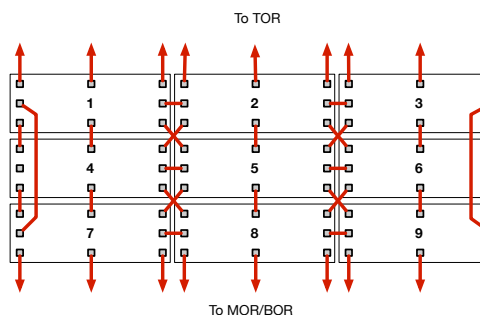
[Q23] What other configurations are possible?

[A23] Here's what a similar configuration looks like with 16 *full* bricks in a 19" rack. Full-brick configurations could come in 8, 16, 24, 32 cell physical clusters. Red links are short cables (Cat 8/RJ45), all running at 25Gb/s. Each server has 8-12 ports (single Netronome chip) connecting to adjacent servers within the rack, and to the outside world through conventional TOR switches.



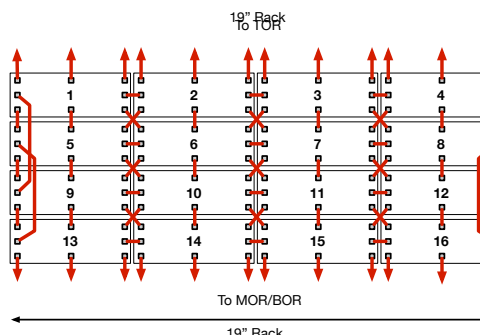
[Q24] Can you use the Facebook OCP 2U3N configuration?

[A24] Yes. Here's what that looks like fully wired within a single rack. OCP 2U3N configurations come in natural 9, 12, 15, 18 cell clusters (2U height, 3 servers (Cells) across). Tiled Tree Paxos configurations make cell 5 the natural (default) master for this tiled configuration, and cell 8 would be chosen (by cell 9) to be preferred failover should it fail. In this way, the The DAE Tiled Tree Paxos (TTP) can thus failover from Cell 5's tree, but still has a full 9 cells (including itself).



[Q25] Can you use the LinkedIn Open19 1U4N configuration?

[A25] Yes. Here's what 1U4N looks like with 16 Half-Bricks wired within a single Open19 rack. Half-brick configurations could come in 16, 32, 48, 64 or 96 cell physical clusters (1U height, 4 half bricks across). A typical intra-rack configuration to exploit the Transaction Fabrix architecture. Tiled Tree Paxos configurations for cells 6 and 7 are failover partners, maintaining 1-hop consensus integrity.



[Q26] What would look like with regular 1U 1N Pizza boxes?

[A26] (See following page, and answer to Kleinberg links.)

[Q27] Why can't you just use faster links, e.g. 40 or 100Gbit ports to unclog the pipeline?

[A27] Because we are running up against the limit for how much we can push through a single port connected through PCIe on an Intel processor. See FAQ-Bandwidth: [Dark Packets](#)¹².

[Q28] What about 100 and 200 Gbit interfaces that are coming this year?

[A28] These new standards use [QSFP56-DD](#), which employs 8 lanes with 25Gb/s NRZ modulation or 50Gb/s PAM4 modulation. This is perfect for our Transaction Fabrix. Instead of trying to get 200Gbits into a server on a single port (which suffers from the dark packets problem mentioned above), we use each of the 8 lanes for independent ports into each server. Not only does this provide a more usable 25Gbit/s into each port without the dark packets problem, it also provides a cell valency of 8 for our dense fabrix, partition-resistant, tree based addressing system, for vastly more reliable Paxos consensus and transaction frameworks.

¹²Above 40Gbps, we see [Dark Packets](#), due to the gap between cache and DRAM latency. At 100GbE almost all packet sizes are under the latency line. So we can't get more bandwidth in and out of a server unless we go to more ports. Cells in the TF have 8+ ports. By operating each port in the range 25Gbps, we maximize the total throughput of a multi-core cell, or more interestingly, Cells can offer forwarding services via the NIC without involving the main (Intel) processor.

[Q29] What do Kleinberg links look like with 1U servers (1U1N) ?

[A29] Within a rack, cables are short. Direct connections (between adjacent cells are $\sim$ the pitch between servers (1.75 inches $\sim$ 6 cm¹³): Kleinberg links stretch between 2U $\sim$ 6 cm, 4U $\sim$ 12 cm, 8U $\sim$ 25 cm and 16U $\sim$ 50 cm distances between cells within a rack.

[A29] Between racks, the cables may be physical (limited to 8m for low insertion loss), or they may be ‘logical’ Kleinberg links (that fast forward through special pairs of ports in the SmartNIC). Either way they resemble the figure (physical or logical) to the right.

[Q30] So your minimum cable hopcount is 6 or 7?

[A30] No, it’s 1 (short one). In a Clos it’s 2 plus the TOR delay.

[Q31] So then your average hopcount is 6 or 7?

[A31] No, it’s 1. Maybe 2 in cases where the cell is used as a load balancer for high fanouts, or a logical dispatcher to many adjacent cells for deep learning configurations.

[Q32] Why do you refer to the Clos Network as a ‘choke point’?

[A32] Rather than squeeze all traffic through a Clos network, a Transaction Fabrix has many more alternative paths.

[A32] For an 800 cell/sever comparison:

- There are 3022 links in the Transaction Fabrix, all of them available for functional packets.
- There are 800 links in the Clos for functional packets, the remainder are transit, and must be higher bandwidth.

[Q33] Why not use flat addressing like a Clos network?

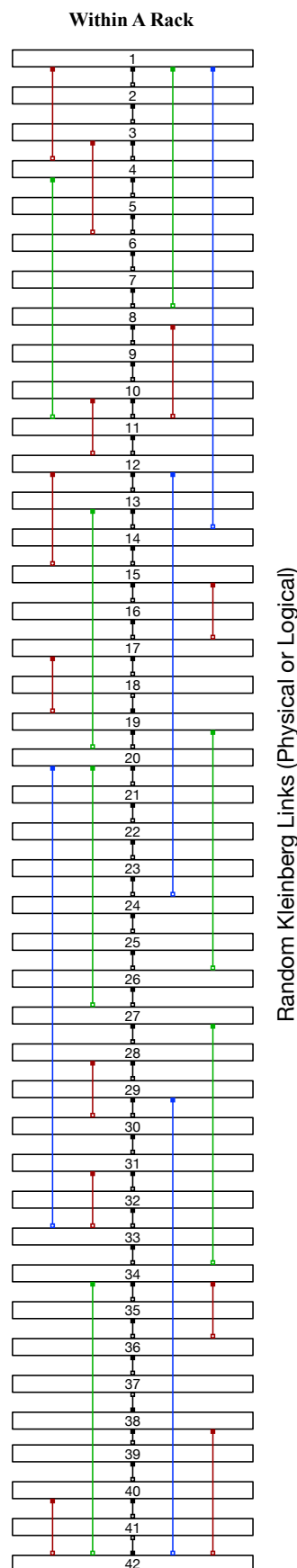
[A33] Clos networks build spanning tree on the switches to give the servers a view of a flat network. That’s a bug not a feature. A flat network allows any node to address any other node, which adds both complexity and vulnerability.

[A33] Flat addressing implies that all servers are equally distant from one another in terms of hopcount and latency. I.e. 6 or 8 cable hops (plus 5-7 switch hops). The assumption is that servers can be placed *randomly* anywhere, making them easier to manage.

[Q34] What’s wrong with that assumption of random placement?

[A34] 75% of traffic is already within a single rack in many datacenters. For a Clos Network, two cells communicating with each other, even within a single rack, must go through two 3-5m cables and a Top of Rack (ToR) ($\sim$ 280ns) switch, for a total of around 300ns *unnecessary* latency. The Transaction Fabrix replaces this inefficient connectivity with single 0.2m cables between cells (3ns), for a two orders of magnitude improvement in link latency.

¹³The top connector for the self cell connects to the bottom connector of the cell above; the bottom connector connects to the top connector of the server below.



[Q35] What is ‘Tiled Tree Paxos’?

[A35] TTP is a special version of the Paxos consensus algorithm which exploits the DAE protocols. This is the same as ‘consensus tiles’ describe in other FAQs. It’s advantage is that cluster membership is known instantly (on link failure). The master cell is self-selected at boot time, and it can preselect its preferred proxy to take over in case of failure. If the preferred proxy fails, then any of the cells 1-hop away can be selected (by the algorithm) to become the new master.

[A35] Every cell is the center (master) of its own TTP Cluster, on its own tree. This allows ‘tiling’ of the consensus clusters across and up and down the rack. The mechanism is intrinsic to and available for every cell (when connected to neighbors), although it can be turned off, or enabled through an API to the Virtual Machines and applications above.

[A35] TTP is (theoretically and practically) the most reliable consensus mechanism that exists today. It relies on the DAE protocols in a switchless environment, with a single cable connecting between servers at 25G/50G or 100G between cells (servers).

[Q36] On Paxos ... why are 9 nodes an important cluster size?

[A36] Paxos (like any consensus protocol, such as Zookeeper or Raft), requires $N/2 + 1$ for a majority quorum. Paxos performance depends on the number of nodes in the cluster. $N = 3, 5$ is conventional wisdom (for Clos networks), and $N = 7, 9$ is considered to be too slow. A [simple analysis](#), shows that it’s not just the network delays and uncertainty, but also that the pipeline gets clogged – i.e. the number of concurrent ports that a master can broadcast on and receive replies. When we compare a single port Clos Network (e.g. what Facebook uses), with the Transaction Fabrix all ports can be activated concurrently to send and receive messages. Single hop fabrics such as the Transaction Fabrix not only dramatically improve performance, they also enjoy ‘common knowledge’ of the cluster membership. This represents a substantial improvement in resilience; which is not possible in a Clos network implemented with DReDDful¹⁴ switches.

[Q37] Don’t you need multiple endpoints with separate addresses to manage things like failover?

[A37] No. With Tree-Based Addressing, you can have any number of endpoints on the tree be guaranteed to receive all packets sent to that ‘tree’. The foundational primitive in the protocol is **treecast** (all members of the set on the tree), so you can have any amount of redundancy on the tree, plus the tree can grow and shrink in real time to accommodate things like recovery from failures, and varying traffic load, without requiring an N^2 scalability penalty.

[Q38] How many hops are in a large datacenter?

[A38] In Transaction Fabrix, each hop does something, rather than being simply a packet forwarder. For example, a typical microservices infrastructure would have an ingest stage that would be a load balancer a 2nd stage a firewall, 3rd stage for Web servers, 4th the Application stage, 5th the Database, etc. This is the distinction between functional traffic, and transit traffic described previously.

[A38] In many applications (such as Deep Learning) the Transaction Fabrix supports different kinds of hops. Virtual Kleinberg links, for example, do not need to go into the main processor memory space, and can be forwarded by the NIC directly. Other packets can be forwarded by the device driver (DMA to cache operations) and manage tree operations.

[Q39] What if some functions cannot talk directly with 1-hop (logically adjacent) relationships?

[A39] Cells have several fast forwarding modes to accommodate transit traffic. As shown in the figure below, with 8 direct links and 2 [Kleinberg links](#) we can reach several thousand cells within six hops. We think this is sufficient (for now) for a rack-scale interconnect. However, with Kleinberg links and out to 8 hoops, we can reach many more cells, effectively making the Transaction Fabrix scale-independent.

¹⁴Switches are described as ‘DReDDful’ because they Drop, REorder, Delay and Duplicate packets

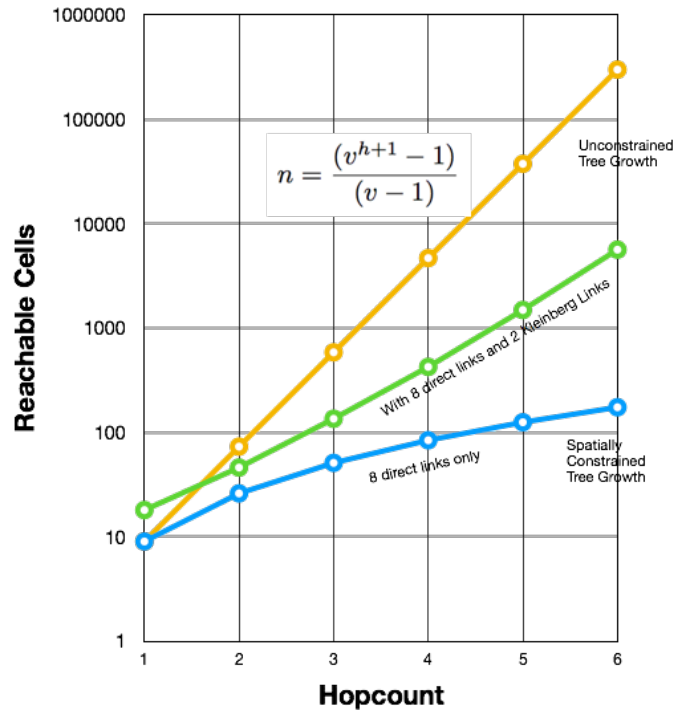


Figure 1: Cell Reachability: Unconstrained, Spatially Constrained, and with 2 Kleinberg Links

[Q40] What about scalability?

[A40] Kleinberg links (physical or virtual) enables [small-worlds](#) networks (an often counter-intuitive result). Clos Networks have scale limitations, for example 2-tier networks are limited to around 1000 servers, 3 tier to 10,000 servers and web-scale companies need to go to hundreds of thousands of servers, which requires yet more tiers. This depends on the radix of the switches. High radix witches have fewer hops, but are more expensive, represent a Single Point of Failure (SPoF) and have a greater blast radius when they do fail.

[A40] The maximum number of hops in a (so called flat) Clos Network:

- 2-tier Clos – 4 hops
- 3-tier Clos – 6 hops
- 4-tier Clos – 8 hops (e.g. Facebook: TOR-Fabric, Spine, Edge))

The first two (2 and 3 tier Clos networks) are for small datacenters, where the number of servers supported $N = P^3/3$ (for 3-tier). 2 and 3 Tier Clos networks are common in small datacenters [ref: Amin Vahdat]. The latter 4-6 tier Clos networks are more common in Web-scale datacenters (e.g. Facebook, Google, Amazon)¹⁵.

[A40] In a Small-World's Network, the average hopcount is $\text{LogLog}(N)$, and the maximum is $\log(N)$ [ref] – which we overcome by dynamic migration.

[A40] Clos Networks also require 'cable management' and 'cable verification', which is not necessary in the Transaction Fabrix because (a) cables are so short, *one* human being can plug both ends of a cable in to adjacent servers and intuitively see they are correct. And (b) we don't care – our algorithms will discover what's there, and adapt to whatever topology is provided (See Daedaelus Product Brochure-Database).

¹⁵In a Small-Worlds random network, it is more like 5.2 than six. The average hopcount for Facebook's graph is closer to 4 (for Facebook). 4.7 for Azure DC network, which is closest to our application.

[Q41] How does Transaction Fabrix compare with switch latency?

[A41] Latency varies considerably for current interconnects: For 40Gbit Ethernet, it varies from an average of 280ns¹⁶, to a maximum of 97,980ns (yes, that's 97.8 μ s) (see: [Tolly Test Report: Mellanox, Broadcom/Arista](#)). For Infiniband, port-to-port hop latencies are on the order of 170 nanoseconds to 220 nanoseconds depending on the protocol and workload ([Next Platform](#)). The GenZ consortium claims that small radix switches should deliver 10-30 ns latency and large radix switches 40-60 ns latency; *these would be state of the art switches*.

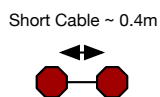


Figure 2: Single functional hop in a Transaction Fabrix $\sim 2ns$

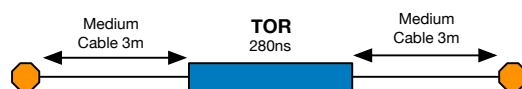


Figure 3: Single functional hop in a Clos $\sim 300ns$

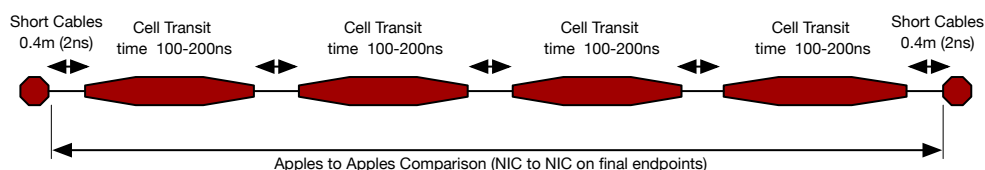


Figure 4: 4 Cell hops in a Transaction Fabrix $\sim 1us$

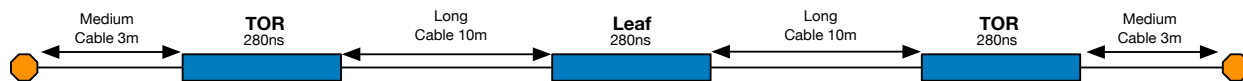


Figure 5: 3 switch hops in a Clos $\sim 1us$

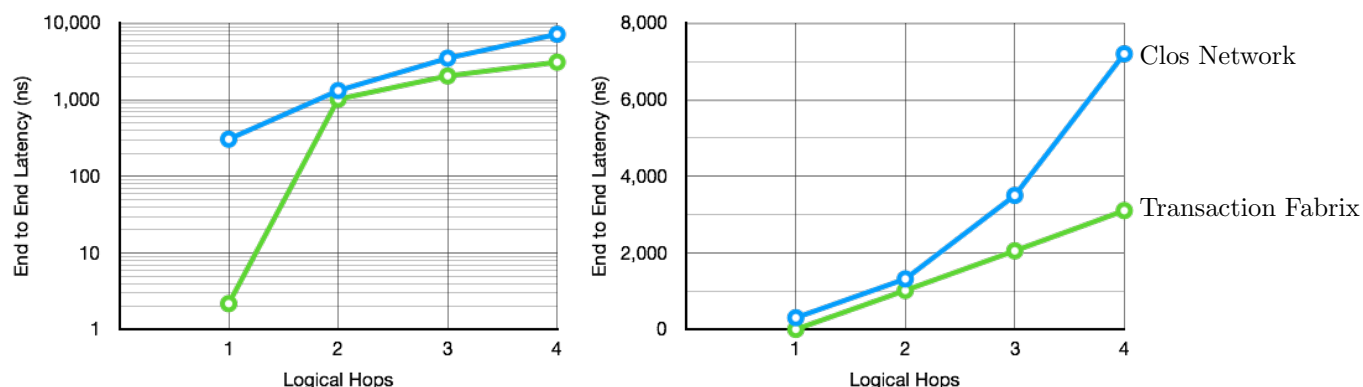


Figure 6: Latency, from a minimum logical hop of 1, for a Transaction Fabrix and Clos, Log and Linear scales
Note that one hop in a Transaction Fabrix presents a major improvement in all aspects (latency $\sim 2ns$, zero packet loss, zero congestion (on that link¹⁷), lower CapEx, lowerOpEx, etc.). This extraordinary result is

¹⁶The minimum latency in a Cisco 5000 series switch is 3.2 μ s, [Arista](#) is 500+ns.

¹⁷By definition, packets are taken off the wire at the same rate they are added.

because one hop in a Transaction Fabrix is simply a *single* cable delay¹⁸, whereas a conventional switched network requires at least one switch (the ToR), and two medium length cables^{19,20}

[Q42] What about layer 1 or physical layer switches?

[A42] [Layer 1 switches](#) are devices that operate on Ethernet links at the Physical layer and therefore switch streams statically, based upon pre-configured rules rather than dynamically, based upon addresses in Ethernet frames. Our forwarding plane includes what we call Huang links, which do layer 1 switching in the same ways as a Layer 1 switch.

[A42] Physical layer switches act like a piece of wire. They can do a little more than that. They typically have circuitry that regenerates the signal (a repeater function). The reason to regenerate is to remove noise from the incoming signal and start again with a fresh clean signal on the output. That is, the individual output bits are replicas of the original input bits where the bits were first generated. This process is called clock and data recovery, CDR or retiming and is a standard part of all network switches.

[A42] The problem with Layer 1 switches is they present additional devices and longer cables in the path that add latency and unreliability. If they use preconfigured (i.e. static) rules, then why not connect the servers to each other directly? i.e. cable them and let the configuration rules be in the SmartNIC's (driven by our Graph Virtual Machine (GVM)).

[A42] Layer 1 switches don't have buffers or queues. They use crosspoint switches, which do not allow for the connection of multiple inputs to a single output, that is, they do not allow packets from multiple links to be aggregated into a single link. (sometimes called muxing or multiplexing). This means that there is hence no need to buffer any packets in a queue that might arrive at the same time.

[A42] Another benefit of crosspoints is that they can connect a single input to more than one output at a time without introducing any overhead. This means that a physical layer crosspoint switch can broadcast data to many ports at the same time. The DAE SmartNIC's on the other hand can multicast, based on a forwarding table set up by the Cell Agent. This multicast is based on a single entry in the routing table (the treeID).

[A42] Layer 1 switches, like patch panels, taps, signal regenerators, media converters and switches – all these devices introduce latency overheads, possible points of failure and tie up valuable rack space (and cost more).

[A42] In the Transaction Fabrix, one short (copper) cable connects adjacent computers. With a layer 1 switch, you need two long cables (to/from the TOR), and a switch which no matter how fast it is, will introduce additional latency, unreliability (twice as many cables, therefore twice as many connectors, plus the device itself can fail, or lose power). Most importantly, [Layer 1 switches](#) requires an entirely separate, and architecturally alien, system of monitoring and management tools, which adds significant OpEx to the management of datacenters.

[Q43] What are the advantages do you get from the TF besides eliminating the TOR switch latency?

[A43] We overcome the “common knowledge” problem that plagues distributed systems, consensus and applications such as load balancing. For a detailed discussion, see the [FAQ-ENTL](#).

[Q44] But isn't forwarding in software always going to be slower than hardware?

[A44] Not necessarily. We plan to do all processing for the forwarding and control planes in hardware: in the smart NIC. There is no reason why a future Smart NIC cannot be designed to forward packets at the same speed as a switch (using its TCAM). Furthermore, our unique routing protocol allows for extremely fast forwarding implementations not possible with IP addressing.

¹⁸We calculate from the external connection on the NIC's at each endpoint for an apples to apples comparison.

¹⁹The cables to the ToR vary anywhere from 0.5m to 5m up and down the same rack.

²⁰Note that in [fig.6](#) the Transaction Fabrix has a large advantage in *one* hop within a single rack. Even for all other hop-counts (2 and above), the Transaction Fabrix wins on latency, even though we assume 280 ns as the average delay for a hardware switch, and 500ns as the average delay through our forwarding plane in a cell.

[A44] Not necessarily, the processing time for packets in software can now approach the speed of dedicated hardware, using the [Data Plane Development Kit](#) (DPDK) with the [Poll Mode Driver](#) (PMD)²¹. In addition, we have our own proprietary way of dramatically reducing the number of instructions required for a lookup: typically 500 machine cycles for a hash function, which because of our unique routing method, we can achieve in 10-20 instructions (100-200 ns in the device driver).

[Q45] What about Cut-Through Switching? Aren't switches better at this than a typical Smart-NIC?

[A45] [Cut-Through Switching is a Myth](#). Cut through routing/switching works only when the port speeds are the same for the incoming and outgoing ports. This is why our initial product uses 25Gbit everywhere: ingest port bandwidth is the same as egress port bandwidth (particularly for Huang links)²².

[A45] Cut-through is when a switch (or NIC) starts forwarding a packet as soon as the header is received. The idea is to reduce latency when forwarding transit packets. Store and forward, in comparison, requires buffering the entire incoming packet (and computing a checksum) before forwarding it to an output port. The performance of many application infrastructures, are sensitive to the latency of control packets, which are typically short (used for synchronization). We discuss the packet length in the next section.

[A45] A packet can cut-through only if the egress port is empty. Even networks with low utilization – as little as a few percent – are not able to use cut-through in most cases, and would not permit the establishing of deterministic low-latency bound that cut-through switching allegedly guarantees²³.

[A45] Typical datacenter networks are multi-tier, multiplying delay with the number of hops. This is why system vendors focus on building large, strictly non-blocking switches. If the switch is not strictly non-blocking (e.g., XAUI based switch), the probability for a packet to be unnecessarily delayed multiplies, not just due to intermittent oversubscription to one port, but also due to artificial (i.e., blocking) oversubscription. As a result, the original drive to provide cut-through switching actually produces increased delay and latency, contradicting the objective.

[Q46] But what about long packets, and the bit transfer time on the wire?

[A46] That's a bandwidth argument (which we will deal with shortly from a latency perspective). This FAQ is about latency (that aspect of performance governed by the laws of physics).

[Latency trumps bandwidth](#) in modern microservice application infrastructures.

[A46] A focus on bandwidth (faster network ports, longer packets to amortize the headers) is the obsession of inexperienced engineers. The hard problem (which we have solved) is the optimization of 'interactivity' or 'temporal-intimacy' in distributed systems. This is a more sophisticated concept than simple latency, which is often misunderstood because people only consider the minimum or average latency. See : [Preventing Long-Tail Latency](#), [Gil Tene Blog](#) and [Gil Tene Video](#).

[Q47] What if 'something unexpected' in provisioning functions on the fabric require multiple hops?

[A47] Then you get gridlock like having too many cars in a City like [New York](#) or [Paris](#). Thought experiments that push resource exhaustion in [networks](#) as an engineering goal are the wrong way to look at things, and this kind of thinking is a failed architectural theory in networking. The Transaction Fabric is not the internet. It does not drop important packets, especially not the small packets that represent transactions or consensus.

[A47] Then configure services (such as logging) which are not latency sensitive, and do not have high 'temporal integrity' needs to perform their function.

²¹see: [Analysis of performance of Intel DPDK on Physical and Virtual Machines](#)

²²At 40Gb per link and above, it is better to have more ports than to try and stuff more packets into individual faster ports because of Dark Packets (see related question below.)

²³Packets between a few ports of the same speed are store-and-forward even in cut-through enabled switches.

[Q48] What size packets are needed for transactions?

[A48] Blockchain transactions (for example) are typically 250 bytes. Packets in consensus protocols can often be smaller than that (e.g. 64 bytes or less).

[Q49] What is Tail Latency?

[A49] The emphasis on latency, instead of just throughput, puts increased pressure on system designs that improve both parallelism and single-thread performance. See [Amdahl's Law for Tail Latency](#).

[A49] A major architectural breakthrough in the Transaction Fabrix architecture, is the emphasis on 'temporal intimacy' (TI). TI provides a significant benefit to systems because it makes them faster and more responsive. We now know from experience with large systems that small (1-2) valencies on servers, separately switched networks that drop, reorder, delay and duplicate packets create undesirable dynamics of cascading failures as systems scale.

[A49] The Transaction Fabrix not only provides lower minimum latency than a Clos Network (eliminates TOR delays, and most transfers are one hop), it also eliminates long-tail latencies.

[Q50] Why are you so obsessed with microseconds (as opposed to nanoseconds or milliseconds?)

[A50] We are obsessed with the optimization of 'temporal intimacy'. We believe this will have a far more significant impact on the latency, resilience and security of systems built on the Transaction Fabrix, than any of the traditional thought processes that lead to longer packets, more buffers or any of the typical bandwidth optimization practices in networks today.

[A50] You can go 1 Kilometer in 3.3 microseconds. Microseconds are important. See: [Taming the long latency – when More Machines Equals Worse Results](#), and [Jeff Dean and Luiz Barroso: The tail at scale](#).

[Q51] How do you calculate the wire propagation delay exactly?

[A51] See Fig. 7 – The speed of light in:

- Free space is 2.998×10^8 m/s. Copper is 2.3×10^8 m/s, Fiber is 2.0×10^8 m/s ²⁴.
- Not only is the speed of light slower in fiber additional latencies occur in the electronics required to convert from electrical to optics, and visa versa (the MAC conversion interface – SFP+ and QSFP) ²⁵.

[Q52] Doesn't the low valency of a cell limit the reachability of cells compared to a high radix switch?

[A52] Reachability (number of nodes vs hopcount) for a general tree (unconstrained by physical packing) is:

$$n = \frac{(v^{h+1} - 1)}{(v - 1)} \quad (2)$$

See: Figure 1: In the same number of hops as a *flat* Clos network (6-7) we can reach almost 300,000 cells (unconstrained physically by neighbors – i.e., we can use Kleinberg Links for long reach).

[A52] Note: a linear graph of an exponential will often behave this way, Fig. 1 illustrates how reachability improves with a hop count, when we have a valency ($v = 8$). The same effect occurs if we keep the depth constant, and increase v through the range 6-10.

[A52] Combine the cable delays from Fig. 7, the switch delays from Fig. ??, and the Packet Transmission times from Fig. 8 and we have a complete picture of end to end latency.

²⁴See: [Light travel through a fibre optic cable](#)

²⁵This happens in some copper (Twinax) situations also, but only for longer cables; the SFP electronics for short connections is a straight-through electrical connection. However, there are some situations, such as the Intel 82555 controller where the PHY has been designed to be off chip. This is an anomaly. Long copper cables may require off-chip PHY's, but short copper cables do not. See FAQ-Hardware for details.

Constants	Mul/divisor	SFP	Free Space	Copper	Fiber
Speed of Light	c		2.998E+08	2.3E+08	2.0E+08
1 meter travel time	1/c		3.336E-09	4.3E-09	5.0E-09
1 meter (ns)	1000000000		3.34	4.35	5.00
Clos	Length (m)				
1 Clos Physical Link through TOR	3	Yes		13.04	15.00
1 Clos Physical Link TOR-Leaf	10	Yes		43.48	50.00
1 Clos Physical Link Leaf-Spine	30	Yes		130.43	150.00
1 Clos Physical Link Spine-Gateway	50	Yes		217.39	250.00
Cellular Fabric	Length (m)				
Adjacent Computer	0.1	No		0.43	0.50
Adjacent Rack	1	No		4.35	5.00
Short Kleinberg	3	No		13.04	15.00
Medium Kleinberg	5	No		21.74	25.00
Long Kleinberg	10	No		43.48	50.00

Figure 7: Cable Propagation Delay

[Q53] How long does it take to send a packet?

[A53] See Fig. 8. (Next page).

Packet Transmission Times in ns

Total Overhead	Packet Size	Total (Bytes)	Total (Bits)	Bits after encoding	Time on Wire [Gbit/s] / (ns)				
40				1.03125	25	50	100	200	400
40	64	104	832	858	34.32	17.16	8.58	4.290	2.145
40	128	168	1344	1386	55.44	27.72	13.86	6.930	3.465
40	256	296	2368	2442	97.68	48.84	24.42	12.210	6.105
40	512	552	4416	4554	182.16	91.08	45.54	22.770	11.385
40	1024	1064	8512	8778	351.12	175.56	87.78	43.890	21.945
40	1280	1320	10560	10890	435.60	217.80	108.90	54.450	27.225
40	1518	1558	12464	12853.5	514.14	257.07	128.54	64.268	32.134

Figure 8: Ethernet packet transmission times from 25Gb/s to 400 GB/s

[A53] While 25-400 Gbit (Fig. 8) Ethernet is our initial implementation choice, our mechanism (bits + protocols) can be added to any protocol (Infiniband, Omnipath, GenZ, or any future Ethernet, etc.).

Ethernet Frame

Ethernet	Pre.	SOF	Dest	Src	Tag	Ether/Len	Payload	CRC	Gap	Total	Overhead
Bytes	7	1	6	6	4	2	46	4	12	88	40
Bits	56	8	48	48	32	16	368	32	96	704	
% of Total	8%	1.1%	6.8%	6.8%	4.5%	2.3%	52.3%	4.5%	13.6%		45.5%

Figure 9: Ethernet Frame format

[A53] The bit transmission times for a small Ethernet packets shrink significantly (from 34.32ns to 2.15 ns) as the Bit rate goes from 25Gb/s to 400 Gb/s. These are increasingly dominated by the cable delays in a Clos (10's to 100's of ns). Compare this to the Transaction Fabrix, where most cables are direct, and less than a tenth of a meter, especially in a Chiplet Motherboard environment. ((2ns). Even the physical Kleinberg links are still short enough to use inexpensive connectors and copper cables (see discussion on physical and logical Kleinberg links – in FAQ-Networks).

[Q54] Doesn't your Liveness (ENTL) packets interfere with normal packet transmission?

[A54] Not at all. When the link is idle, mutual state coupling is maintained silently through the DAE Non-Time Liveness (ENTL) protocol. When application packets are being sent, they are piggybacked in other places, such as the preamble, disparity codes, header or other fields. Besides, we only need a minimum number of bits to maintain the event/temporal-intimacy (theoretically, only 4 bits, see FAQ-ENTL).

[Q55] If temporal intimacy requires a small amount of information, why use a whole packet?

[A55] Because we are limited by the size of Ethernet Packets. When our protocols are on other rack scale interconnects (for example, PCIe), they can be as small as 4 bits (although 4 bytes, 64 bytes, and 128 bytes (the size of the x86 cache line pair) are particularly useful for communicating with the main processor.

[Q56] What are Huang Links?

[A56] Huang links are direct pass through links from a single (bidirectional) ingress port to a single egress port. A 'logical' Kleinberg link is composed of one or more Huang links, to reach through an arbitrary number of cells with the lowest possible latency. Huang links can be forwarded in software (in the Main device driver, the NIC firmware, or in specialized NIC hardware), or in a special hardware bypass programmed by our software, which turns them on or off and configures them. Huang links are *logical* when they are forwarded at the lowest level of the Network Asset Layer (NAL) and *physical* when [bypassed by relays](#) that directly connect the electrical signals.

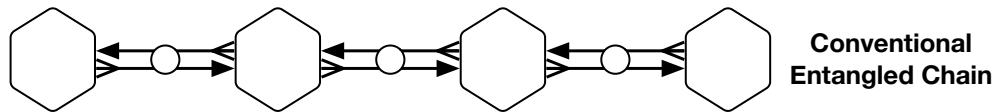
[A56] Huang links are a particularly important optimization: they offer simple but very high performance 'cut-through' forwarding²⁶; because: (a) the header does not need to be decoded before the bits, bytes or packet(s) are immediately forwarded (b) the output (egress) port is always guaranteed to be free (because it cannot be used by any of the other trees)²⁷, and (c) the atomicity of the The DAE Non-Time Protocols (ENTL and ENTT) protocols can be guaranteed as they go through the Huang links without being interfered with in any way (including eavesdropped) by the local cell²⁸. Huang links can be physical only to the limit of the insertion loss of the link. However 'logical' Huang links are not because their signal is regenerated by each repeater.

[A56] Another advantage of the Huang links, is that they are bidirectional, and the back-pressure if buffers are full in the destination is returned immediately as fast as physically possible through the same links. This

²⁶depending on the NIC. In principle, future NIC designs can achieve full cut-through performance.

²⁷This is only true In the case of a Huang link which for the entire chain, acts like a single in-order functional link.

²⁸It is important to note that the local electronics provides a signal regeneration capability which provides long-reach accessibility to other cells, while still maintaining the short physical transmission lines and low insertion loss.



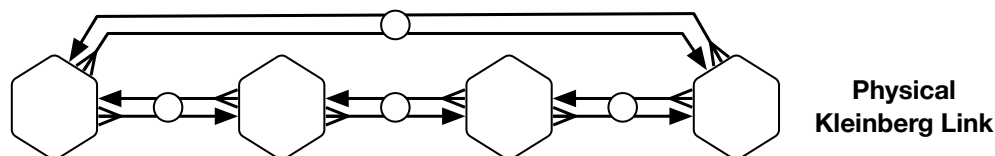
provides both back pressure and in-order delivery, but no buffering in the path beyond a single packet. This in-turn helps protect the integrity of the DAE Non-Time Transaction (ENTT) protocols across a chain.

[Q57] What is the use case for these Kleinberg/Huang Links?

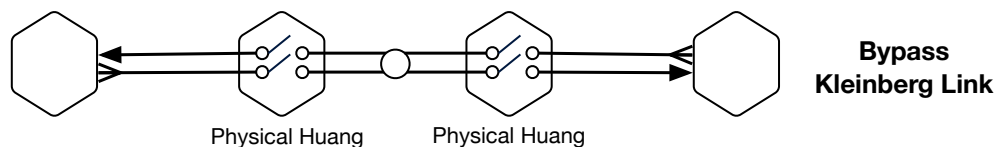
[A57] There are many reasons to use ultrafast long links, a few examples include:

- To connect Consensus Cliques to a more distant ones for failure independence.
- For richer logical connectivity for Deep Neural Network (DNN) hidden layers, or complete [Recurrent Neural Network](#) (RNN)'s (Hopfield, Elman, Jordan, etc.), [Long-Short-Term Memory](#) (LSTM) relationships, etc.
- For faster fanout (reachability) beyond the constraints of immediate physical neighbors. E.g. Load balancers between ingest, web-tiers, app tiers, database shards, etc.
- For virtual Baran links, to enable various graph topologies connected by different logical topologies (such as hypercube, torus, etc.).
- For secure underlay links to different parts of the same tenant or security domain.
- To provide a control plane 'underneath' stacked TRAPH boundaries for the datacenter manager, or tenant manager to hierarchically organize and control the resources they are responsible for.

For reference: a conventional chain of cells and ENTL links. The normal situation for the physical (and logical) interconnect. Note that physical cells will have 8-12 ports. Shown here are the 6 logical ports presented in the logical datacenter display.



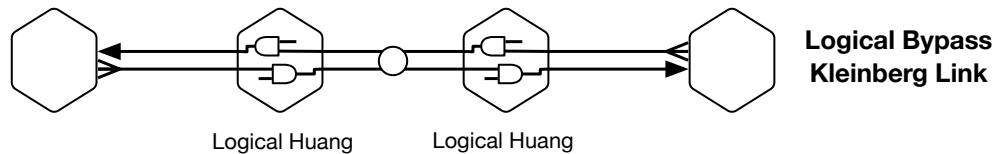
A Physical Kleinberg Links (PKL) may literally be an additional physical cable. They are shown in the logical datacenter display as ordinary links. PKL's provide a small-worlds effect (six degrees of separation for any size network)²⁹.



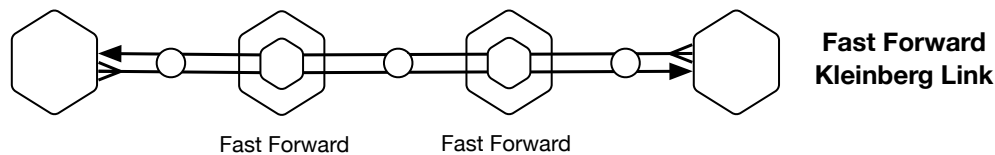
Bypass Kleinberg Links (BKL)'s are another variation on a physical Kleinberg Links, where the cell is either down and the relays fail closed, or software is used to provide a long term direct connection between a single input port and a single output port. We call this a Huang link³⁰. Multiple Huang links make up a single physical Kleinberg Link. These links have electrical (insertion loss) limitations and may be able to bypass only a few cells at higher speeds (50Gbit and above), unless the relays are designed to have a uniform characteristic impedance going through them.

²⁹In practice, we achieve less than six hops within a single datacenter row of racks (40 cells in 20 racks = 800 cells per row)

³⁰Huang links are named after Alan Huang who discovered how to reduce cross country latency in a routed network by literally bypassing the router with direct physical (fiber) connections.



Logical Kleinberg Link (LKK)'s are implemented entirely as a programmable function within the NIC. They can be turned on (to connect exactly one input port to one output port), or off (so they flow through the cell forwarding engine as normal). LKK's do not have the electrical limitations of BKL's and can go any arbitrary distance, because the logic gate provides an electrical repeater function. The ENTL and ENTT tokens may pass along the entire Kleinberg link and back forming a single deep asynchronous timing domain.



Fast Forward Kleinberg Links are functionally equivalent to the above, i.e., one input port is connected to one output port (i.e. no 'up-tree', but each link has its own ENTL token, and "awareness" of the ENTL chain is maintained by passing a decorated ENTL packet down the entire logical link and back to maintain liveness.

[Q58] How can you forward a packet through a cell faster than a switch can?

[A58] First, any technique a hardware switch can use, we can theoretically use, because the Smart-NIC is implemented in a single chip, with approximately the same technology considerations.

[A58] Second, we programmatically tie ports together (Huang Links described above), for three different kinds of *fast forwarding*:

1. Relays: one or two "pairs" of ports may be used to bypass the chip altogether. It's as if there was a straight wire connecting the input port to an output port. There won't be an electrical advantage of signal regeneration, but there will be no congestion (the output terminal is always available), no buffering (no unnecessary mechanism adding latency), and no complexity (the input terminal is tied to the output terminal with no unnecessary mechanism).
2. Gate (logic) connection. The Smart-NIC chip can be programmed to do an 'electronic bypass', on one or more ports. This is a software configuration, which works in conjunction with the Transaction Fabrix architecture.
3. Fast firmware forwarding. Within the forwarding engine on the Smart-NIC, there is a mode, identifiable through the header bits in the packet, which identify that port to port (without header processing) can take place, because the output port is implicitly (for this port) or otherwise predefined.
4. Fast Tree Forwarding. Because of our unique addressing mechanisms, the forwarding engine can forward using an index into a table, instead of a hash. This substantially reduces the per-packet processing time for many packets.

[Q59] What about PingMesh?

[A59] Microsoft's [Pingmesh](#) provides an excellent contrast between conventional networks (which Pingmesh serves) and the DAE Transaction Fabrix approach.

The two separate worlds of server and network administration has driven the evolution of two different cultures, each with different tools, different ways of understanding, and goals that are not always aligned. Ping-mesh is a response to the lack of visibility in the network devices. SNMP is not always reliable, and the

application teams just want to know "why the network appears slow"? This separation of cultures creates many opportunities for finger-pointing and the need for ever-more complex and expensive tools to isolate problems and help debug issues.

The Pingmesh paper contains many examples of the cognitive complexity and error-prone nature of managing networks, and the difficulty of transparently locating failures. They measure latency from an application perspective. Pingmesh collects tens of terabytes of latency data per day. They use machine learning over the data to gain insights into the latency behavior of the network.

Cognitive complexity (Unresolved uncertainty): "These distributed services are large and evolving software systems with many components and have complex dependencies. All of these services are distributed and many of their components need to interact via the network either within a data center or across different data centers. In such large systems, software and hardware failures are the norm rather than the exception. As a result, the network team faces several challenges.

The first challenge is to determine if an issue is a network issue or not. Due to the distributed systems nature, many failures show as 'network' problems, e.g., some components can only be reached intermittently, or the end-to-end latency shows a sudden increase at the 99th percentile, the network throughput degrades from 20MB/s per server to less than 5MB/s. Our experience showed that about 50% of these 'network' problems are not caused by the network. However it is not easy to tell if a 'network' problem is indeed caused by network failures or not."

The Transaction Fabrix (TF) resolves this uncertainty. It doesn't need external tools to measure or understand the latency behavior of the communication fabric. Because of the unique 'tree addressing' structure employed in the TF, developers can directly use these tools as part of the fabrix. For example, we don't report packet loss to some big monitoring service in the sky. We don't need to. the TF doesn't drop packets. In the event that a hardware or electromagnetic interference error occurs, we turn the dropped or corrupted packet into a link failure, and heal via another link. In the conventional world of networking, this would be considered abhorrent. In our world, this enables us to precisely manage the shared information in each link, and hence resolve the uncertainty of whether information has been lost or not (see the exactly-once problem in FAQ-ENTL).

More importantly, the TF world is one world, one culture, one set of tools. Yes, the developers now have a new tool set: a Tree Programming Interface (TPI) to control, and monitor, and measure, their own communication substrate, from their vantage point. Our hypothesis is that this small increase in capability comes with a vast decrease in overall complexity; and the developers will be willing to learn this tool set because it puts them in control when it comes to building and tearing down the virtual topologies their microservices need to communicate with each other. All within a confined set of resources they have been delegated by the datacenter administrator.

[Q60] What about TIMELY

[A60] [TIMELY: RTT-based Congestion Control for the Datacenter](#) is very similar in intent to the PingMesh approach, but uses special hardware in the SmartNIC (Synchronized clocks) to provide a much finer grained control over the temporal intimacy. Although we are typically unimpressed by attempts to use timestamps, they use them here to measure round-trip latency. RTTs measured by hardware timestamps have a much smaller random variance (because all measurements are made relative to the *local* clock) than that by kernel TCP stack. They use this to model latency, noticing that the Queue Lengths have an uncanny correlation with the measured RTT, which, they claim, can be used as a proxy for the queue occupancy and congestion on links.

[Q61] How do you compare with the Google View (Killer Microseconds)

[A61] In the [Attack of the Killer Microseconds](#) By Luiz Barroso, Mike Marty, David Patterson and Parthasarathy Ranganathan, they talk about the need for new latency-mitigating ideas, including in hardware. Microsecond-scale I/O means tension between performance and productivity.

[A61] DAE is very much about important new ideas to reduce latency. In addition to using an event-only protocol that eliminates the need for heartbeats and retries, we also manage 'common knowledge' for consensus and transactions in the link, 'below the hardware' (or at least on the other side of the PCIe bus which adds

almost a microsecond all by itself when accessing devices on the other side). We also agree with Google: a synchronous programming mode is a lot simpler, hence easier to write, tune, and debug. However

[A61] The EC model can be loosely described as synchronous trees. A root of a tree is able to send out work, and receive answers, while maintaining order. This provides an entirely different dimension on the notion of synchronous. The fundamental primitive is reliable multicast. Our consensus Tiles (one hop cluster), is capable of the highest possible reliable multicast because it transparently recovers from failures well below the ‘temporal intimacy’ of the TCP/IP stacks and the applications.

[Q62] How do you compare with the EPFL View (Killer Microseconds)

[A62] in [How to Measure the Killer Microsecond](#) Mia Primorac, Edouard Bugnion and Katerina Argyraki measure tail latency for different forwarding schemes. They use hardware based timestamp counters as a ground truth. They compare two approaches, and in particular whether commodity servers and NICs, when properly configured, can measure the latency distributions as precisely as specialized hardware.

Their results clearly show the benefit of measuring latency of packet requests and responses, and more generally of remote procedure calls, within the NIC as opposed to CPU. They note that “while many modern NICs support hardware-based time-stamping, the implementation is narrowly tailored to the precise requirements of IEEE 1588 time synchronization. A more flexible implementation, integrating bidirectional time-stamping into arbitrary protocols and packet formats, would be highly beneficial.”

[A62] The EC solution is similar, but based on a very different principle. Instead of each timestamp counter supposedly synchronized to many other clocks, we measure events, and durations ‘can be’ counted from the perspective of each side of a link (NIC to NIC). This allows us to reason about causality against elements in a bipartite relationship over a single link, instead of against a background of time. See [FAQ-ENTL](#) for details.

[Q63] How do you compare to Virtual Synchrony?

[A63] [Virtual Synchrony](#) is a “mechanism or algorithm tackling the atomic multicast problem. Similar to atomic transaction, atomic multicast refers to the scenario that a message is delivered to either all group members or to none at all. Atomic multicast refers to the scenario that a message is delivered to either all group members or to none at all. It is named virtual synchrony because operations are often performed concurrently and multicasts are often issued asynchronously, but algorithms can still be developed and reasoned about using simply, synchronous model.”

[A63] There are significant differences. While the original version of Virtual Synchrony attempted to provide a simple ‘synchronous’ programming model, it was criticized for having far too much overhead which limited performance. EC achieves reliable multicast through its consensus tiles one hop away (don’t be fooled into thinking this is a disadvantage). The EC consensus tiles can effectively put a set of 3, 5, 7 or 9 nodes into ‘superposition’ all waiting for a decision. The resolution of uncertainty in using the ENTL and ENTT protocols provides a level of guarantee for ‘reliable multicast’ that goes well beyond that when using conventional networks which duplicate, reorder, drop and delay packets. The result is far higher reliability, and significantly faster performance. See [FAQ-Database](#) for more details.

[Q64] How do you compare to other Low Latency approaches in the literature?

[A64] We have reviewed the following examples:

- [Cure: Strong semantics meets high availability and low latency](#)

Summary

Many aspects of latency have been considered:

1. The type of the cables. The use of copper (which is faster) rather than fiber.
2. The length of the cables. Connect to adjacent computers instead of to/from a TOR.
3. The number of cables one for the Transaction Fabrix, two for Physical Layer switches (plus the switch latency).
4. How many switching nodes are there? For a Transaction Fabrix there can be as few as *none*.
5. How many cable hops? For a Transaction Fabrix one is the minimum, and even that is a very short cable (2ns)
6. What is the length of the cables. In a Transaction Fabrix the cables are all relatively short; and can thus be copper.
7. What is the bit transfer time? – Determines penalty for store and forward instead of cut-through.
8. What is the maximum diameter of the network? (the Clos is flat, and hence $\sim 6-8$. The Transaction Fabrix is variable, minimum being 1 (when adjacent microservice tiers are close to each other physically). The average is *six* (6) – the same as or less than the Clos network.

Bandwidth tradeoffs have been shown to be the wrong tradeoff in modern datacenters comprised of distributed microservices:

1. A 200 MPH racing car is of no use in a neighborhood where it has to encounter frequent turns or slowdowns. Conventional networks are like 8-lane highways. Put all the traffic on the same (one dimensional) path, and you still get gridlock (long tail latency). Put each lane on a separate path in an (8-dimensional) topology and you get very different behavior.
2. [Linux buffers](#) and [buffered sends](#) are evil.
3. Request parallelism can be used to improve latency. This is particularly important for consensus protocols.
4. Temporal intimacy is more sophisticated than simple latency. Although our Link protocol has lower minimum latency (no switch delays), it is far more important to understand that it truncates *tail latency*, which has a far more dramatic effect on the behavior of the distributed system.

The Tyranny of the Orthodoxy

At any given moment there is an orthodoxy,
a body of ideas which it is assumed that all right-thinking people will accept without question.
It is not exactly forbidden to say this, that or the other, but it is “not done” to say it ...
Anyone who challenges the prevailing orthodoxy finds himself silenced with surprising effectiveness.
A genuinely unfashionable opinion is almost never given a fair hearing,
either in the popular press or in the highbrow periodicals

George Orwell

FAQ: Menu

The DAEDAELUS Frequently Asked Questions (FAQ) Document Set

We collect questions from our team and advisors, and in meetings with customers, collaborators and investors. We provide a thoughtful answer to each independent question, and ask the originator if we have answered their question to their satisfaction, or if they have any additional questions or concerns.

Business

Our Business Model.
Why we will succeed.

Market

Our Market Assessment.
Our \$200B opportunity

Product

Details of our Product Strategy.
Durable Distributed Data

Addressing

Tree-based naming, addressing,
and serverless functions.

HOWDO

Distributed algorithms on
self-healing trees

RAFE

Reliable Address-Free Ethernet
Transaction Fabrix

AIT

Atomic Information Transfer
and Exactly-once semantics

Latency

Clos Latency Comparison
Hops, delays & cut-through

Resilience

How to deal with link and node
failures

Architecture

Architecture principles and
interfaces (API, MPI, TPI)

Legacy

Legacy: Compatibility without
compromise.

Customer

Customer related questions,
use cases

Bandwidth

Clos and Cellular Fabrix
bandwidth comparison

LoadBalancing

Load balancing with link state
decoration

Security

Mechanisms to reduce, or
eliminate, attack surface

Causality

Causal interactions in
distributed systems

Monitoring

Monitoring and
observability questions

Simplicity

Datacenter management &
application orchestration

Cloudplane

Technology background. History
of the Company

Nerdy

Talk Nerdy to me.
Technical Deep dive

Subtime

Our model of time. Event-only
protocols.

Competitive

Competitive Analysis and
Positioning

Network-Management

Management questions
OpEx Questions

Classical

Classical Implementation of
Entanglement

Consistency

Consistency guarantees for
distributed systems

Networks

Key Networking Distinctions to
the Cellular Fabrix

Trees

Why trees are a fundamental
distributed systems primitive

Cost

Clos Cost comparison,
CapEx and OpEx

Physics

Physics questions (signaling,
information theory)

Literature

Literature survey on
complimentary technologies

The Case

Extensive arguments for Cellular
Fabrix in datacenters

Hardware

Hardware comparison with
switched networks

Problem

What problem are we solving?
Contextual Answers

ENTL

Ethernet Non-Time Liveness
and Transaction Protocols

Wired

Example Article about EC
in Wired Magazine Style

[Selected Items Only]

From our FAQ repository